

Edge-Native Retail Intelligence Platform: On-Device Vision Architecture, Multi-Task Pipeline Design, and Edge Implementation

Module 1: Problem Deconstruction and the Architectural Advantage

Indian Retail Operational Bottlenecks Across Tiers

The operational landscape of Indian retail diverges fundamentally from Western retail paradigms, presenting distinct physical, infrastructural, and economic constraints across different commercial tiers.

At the base of the retail pyramid, independent Kirana stores and small-footprint merchants (typically 500 to 1,500 square feet) operate under extreme visual clutter, multi-category inventory stacking, and minimal aisle clearance (often below 0.8 meters). These micro-environments suffer from severe physical occlusions, manual billing bottlenecks, and erratic broadband connectivity with frequent dynamic Internet Protocol (IP) renegotiation.

In Tier-2 and Tier-3 regional supermarkets (spanning 2,000 to 10,000 square feet), floor managers face intense footfall density during concentrated evening hours, inconsistent lighting conditions characterized by low Color Rendering Index (CRI < 70) fluorescent tubes combined with ambient daylight, dynamic floor-level stock displays, and cashier-assisted billing counters that lack automated queue balancing.

Conversely, Tier-1 organized retail chains (exceeding 10,000 square feet) contend with significant shrinkage, planogram compliance failures, and phantom out-of-stock (OOS) occurrences where inventory exists in the enterprise resource planning (ERP) system but remains missing from physical shelves¹. For these large chains, continuous streaming of multi-channel Closed-Circuit Television (CCTV) feeds to centralized cloud infrastructure incurs unsustainable egress bandwidth costs and high operational expenditures².

Operational Metric	Kirana / Unorganized Outlets	Tier-2/3 Regional Supermarkets	Tier-1 Organized Chains
Aisle Clearance	< (Severe visual occlusion)	0.8–1.2 m (Moderate occlusion)	> (Standard open aisles)

Ambient Illumination	Variable (150–400 lux)	Mixed fluorescent (300–600 lux)	Regulated (700–1000 lux)
Network Infrastructure	Consumer 4G / Intermittent DSL	Commercial Broadband (20–50 Mbps)	Dedicated Leased Line / SD-WAN
Shrinkage & Phantom OOS	High (Unrecorded inventory loss)	Significant (8–14% phantom OOS)	Monitored (4–7% planogram error) ¹
Compute Deployment Model	Integrated Low-Power Edge Gateway	Multi-Camera Micro Edge Gateway	Distributed Edge Cluster / Hybrid Cloud

The Technical Divide: Naive Implementations versus Qualcomm Edge Solutions

A significant technical gap exists between the naive architectures commonly assembled by novice hackathon teams and the robust on-device implementations expected by enterprise evaluators. Conventional amateur prototypes route uncompressed Real-Time Streaming Protocol (RTSP) video streams to cloud servers hosting unquantized models (such as standard YOLOv8x running in full 32-bit floating-point precision) or execute unbounded OpenCV loops directly on the host Central Processing Unit (CPU). Such designs suffer from severe thermal throttling, frame buffer overflows, uncontrollable network jitter, and excessive bandwidth costs².

Enterprise-grade edge design, in contrast, requires deep silicon-level awareness. Modern system-on-chip (SoC) platforms require heterogeneous execution across the CPU, Adreno Graphics Processing Unit (GPU), and Hexagon Neural Processing Unit (NPU) or Hexagon Tensor Processor (HTP)⁴. Runtimes such as the Qualcomm Neural Processing Engine (SNPE) and Qualcomm AI Runtime (QAI RT) allow developers to offload tensor computations to specialized vector and tensor accelerators⁶. By processing video frames entirely within volatile system memory and discarding raw pixel buffers immediately following feature extraction, the system enforces strict data privacy, operates within a sub-15W thermal envelope, and delivers deterministic inference latency².

Architectural and Mathematical Formulations

Bandwidth Conservation Model

In a conventional cloud-centric vision architecture with N_{cam} cameras streaming video at an average bitrate R_{video} , the cumulative bandwidth B_{cloud} consumed over operational

duration T is expressed as:

$$B_{\text{cloud}} = \sum_{k=1}^{N_{\text{cam}}} \int_0^T R_{\text{video}}^{(k)}(t) dt$$

For a four-camera installation streaming 1080p video at 30 frames per second (FPS) with an H.264 bitrate of $R_{\text{video}} \approx 4 \text{ Mbps}$ per channel, the aggregate data transfer over a 30-day billing cycle evaluates to:

Under the proposed edge architecture, raw video frames never leave the volatile local memory buffer. The edge gateway transmits only compact, structured JavaScript Object Notation (JSON) telemetry vectors (containing spatial bounding coordinates, tracker identifiers, dwell times, and alert flags) at a telemetry frequency $f_{\text{meta}} = 2 \text{ Hz}$:

$$B_{\text{cloud}} = 4 \times 4 \text{ Mbps} = 16 \text{ Mbps} = 2.0 \text{ MB/s} \approx 5.184 \text{ TB/month}$$

Given an average telemetry payload size $S_{\text{JSON}} = 256 \text{ bytes}$:

$$B_{\text{edge}} = \sum_{k=1}^{N_{\text{cam}}} \int_0^T (S_{\text{JSON}} \cdot f_{\text{meta}}) dt$$

$$B_{\text{edge}} = 4 \times (256 \text{ bytes} \times 2 \text{ s}^{-1}) = 2.048 \text{ KB/s} \approx 5.31 \text{ MB/month}$$

$$\text{Data Reduction Efficiency} = \left(1 - \frac{B_{\text{edge}}}{B_{\text{cloud}}}\right) \times 100\% = \left(1 - \frac{5.31 \times 10^{-3} \text{ GB}}{5184 \text{ GB}}\right) ;$$

Deterministic Edge Latency Model

Total event response latency for an edge-native pipeline versus a cloud-dependent pipeline is

governed by the following delay formulations:

$$L_{\text{edge}} = t_{\text{ingest}} + t_{\text{preprocess}} + t_{\text{infer_NPU}} + t_{\text{track}} + t_{\text{rule_engine}} + t_{\text{ipc}}$$

$$L_{\text{cloud}} = t_{\text{ingest}} + t_{\text{encode}} + t_{\text{uplink_rtt}} + t_{\text{cloud_queue}} + t_{\text{cloud_infer}} + t_{\text{downlink_rtt}}$$

Operating on an integrated NPU yields an inference duration $t_{\text{infer_NPU}} \approx 3.5\text{--}8.0 \text{ ms}$, ensuring a total local latency $L_{\text{edge}} \leq 25 \text{ ms}$ ¹⁰. Conversely, public network uplinks in Tier-2/3 regions introduce significant network jitter ($t_{\text{uplink_rtt}} \approx 80\text{--}450 \text{ ms}$), driving cloud latency $L_{\text{cloud}} > 300\text{--}900 \text{ ms}$, which impairs real-time queue preemption and checkout load balancing.

Privacy-by-Design Compliance

Under the Digital Personal Data Protection (DPDP) Act of 2023, the ingestion and cloud transmission of identifiable biometric data in public spaces creates substantial compliance liabilities³. The edge architecture ensures complete data minimization by restricting frame transformations to volatile direct memory access buffers (DMA-BUF) within system RAM⁹. Raw pixel data is overwritten immediately after tensor generation. The external output consists entirely of anonymized spatial metadata:

$$\mathbf{x}_{\text{meta}} = [\text{Class ID}, x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}, \mathbf{v}_{\text{motion}}, t_{\text{dwell}}]$$

Power and Thermal Budget Constraints

Small-form-factor retail edge gateways must operate reliably without active cooling systems in high ambient temperature environments. The total system power consumption P_{total} must remain within passive thermal dissipation boundaries:

$$P_{\text{total}} = P_{\text{SoC}} + P_{\text{DRAM}} + P_{\text{I/O}} + P_{\text{Peripherals}} \leq 12.0 \text{ W}$$

By offloading tensor multiplications to dedicated Hexagon vector and tensor hardware, the energy consumption per inference cycle (E_{inf}) decreases substantially relative to CPU execution⁵:

$$E_{\text{inf}} = P_{\text{active}} \times t_{\text{inference}}$$

Executing an INT8 network on the Hexagon NPU at an active power draw of 5 W for 6 ms consumes approximately 30 mJ ¹⁰, whereas running the same model on an ARM Cortex-A78 CPU cluster at 8 W for 45 ms consumes 360 mJ ⁴. This twelve-fold reduction in thermal energy per inference prevents thermal throttling and ensures consistent processing throughput.

Module 2: Prior Art, Patents, and Competitive Landscape

Commercial Solutions Analysis

Existing commercial retail intelligence systems fall into three primary categories, each presenting distinct economic and architectural trade-offs:

1. **Amazon Just Walk Out and Ceiling-Grid Systems:** These platforms deploy high-density matrices of ceiling-mounted RGB cameras, time-of-flight depth sensors, and structural shelf-load cells. The resulting multi-modal sensor streams are aggregated on localized server racks or streamed to regional cloud data centers. Total installation and hardware costs routinely exceed $\$150,000$ to $\$500,000$ per retail store, rendering them economically impractical for Indian retail environments.
2. **Traditional Video Management Systems (VMS) with Basic Analytics:** Commercial CCTV deployments utilize central digital video recorders (DVRs) or network video recorders (NVRs) running basic analytical plug-ins on host x86 or ARM CPUs. These legacy systems rely on simple background subtraction or crossing-line triggers. As a result, they suffer from high false-positive rates during lighting transitions, cannot resolve human occlusions, lack planogram-aware inventory tracking, and offer no programmable edge application programming interfaces (APIs).
3. **Cloud-Native Retail Analytics Platforms:** Cloud SaaS providers ingest encrypted H.264/H.265 video feeds from IP cameras over wide-area networks (WAN). These architectures incur heavy recurring cloud infrastructure costs, fail completely during broadband disruptions, and present regulatory compliance risks under emerging data sovereignty and privacy mandates².

System Architecture	Capital Expenditure (per Store)	Operational / Bandwidth Cost	Network Dependency	Occlusion Handling Robustness	Edge Native / Privacy Compliant
---------------------	---------------------------------	------------------------------	--------------------	-------------------------------	---------------------------------

Amazon Just Walk Out	Very High (>)	High (Local Server / Cloud)	High	High (Multi-angle Fusion + Weight)	High (Localized Server)
Cloud Video Analytics (SaaS)	Low (CCTV Ingest)	Very High (\$100–\$300/)	Absolute (> Uplink)	Poor (Frame Drops & Latency)	Low (Raw Video Transmitted) ⁹
Traditional Smart NVRs	Moderate	Zero	Zero (Offline DVR)	Very Poor (Tripwire heuristic)	Moderate (No off-site video)
Proposed Edge-Native Platform	Ultra-Low (<)	Negligible (<)	Zero (Fully Offline Capable)	High (State Persistence Filter)	Absolute (Hardware-level PII Drop) ⁹

Failure Modes of Existing Systems in Indian Retail Environments

Traditional vision platforms struggle when deployed across Indian retail environments due to four interrelated factors:

- Capital Expenditure Sensitivity and Operating Margins:** Indian fast-moving consumer goods (FMCG) retailers operate on narrow gross margins ranging between 2% and 5%. Proprietary hardware installations that demand shelf-embedded load cells or dense multi-camera ceilings cannot generate a viable return on investment.
- Bandwidth Instability and Uplink Degradation:** Commercial broadband connections in Tier-2 and Tier-3 urban centers frequently experience packet loss, dynamic IP reassignment, and carrier throttling. Cloud-reliant computer vision systems suffer dropped frames, resulting in broken object tracking IDs, lost dwell-time measurements, and delayed out-of-stock alerts.
- Packaging Density and SKU Visual Clutter:** Western retail layouts emphasize organized single-facing shelf arrangements. Indian store fixtures feature high-density packaging, hanging multi-tier sachets, unstandardized plastic containers, and multilingual labels that degrade generic object detection networks trained solely on clean benchmark datasets like Common Objects in Context (COCO).
- Illumination Shifts and Optical Glare:** Fluorescent tubes with high flicker rates, open storefronts exposed to shifting sunlight, and deep shadows across lower shelving tiers disrupt standard color-space thresholding and low-dynamic-range camera sensors.

Defensible Innovation and Intellectual Property Framework

To overcome these operational challenges without escalating hardware costs, the platform introduces three core algorithmic innovations that function entirely on edge hardware:

1. **Hierarchical Planogram Geometric Mapping (HPGM):** A lightweight spatial calibration algorithm that maps physical multi-tier shelf structures into normalized two-dimensional polygon coordinate arrays. HPGM isolates inventory regions from non-inventory floor and ceiling pixels before inference, reducing spatial processing overhead by up to **60%**.
2. **Centroid-Shelf Interaction Matrix (CSIM):** A spatial-temporal filtering state machine that continuously monitors intersection vectors between human tracking centroids and shelf region-of-interest (ROI) polygons. When a customer stands in front of a shelf, the system enters an occlusion-locked state, preventing false out-of-stock alerts without requiring secondary camera perspectives¹.
3. **Temporal Multi-Task Pipeline Splitting (TMPS):** An asynchronous scheduling engine that multiplexes person tracking, queue analysis, and inventory depletion across different execution frequencies, aligning computational load with the real-world operational dynamics of each task.

Module 3: Edge-Native System Architecture and Pipeline Design

Comprehensive End-to-End Dataflow Architecture

The edge-native platform executes an end-to-end vision and analytics pipeline entirely within local hardware, ensuring continuous processing without external cloud dependencies:

The camera interface ingests 1080p video streams at 30 FPS over USB Video Class (V4L2) or IP camera RTSP connections via a hardware-accelerated GStreamer ingestion pipeline. Video frames are decoded directly into direct memory access buffers (DMA-BUF), eliminating zero-copy overhead between the camera driver and system memory.

A circular volatile memory ring buffer maintained in low-power DDR4X (LPDDR4X) system memory enforces a strict maximum queue depth of five frames. If downstream inference pipelines experience transient backpressure, older frames are overwritten in place. This mechanism ensures that the neural network evaluates only the most recent frame, preventing queue latency buildup and maintaining deterministic processing times below 33 ms.

The Spatial Region-of-Interest (ROI) filtering module applies predefined polygon masks to incoming frames, removing non-operational visual zones such as ceilings, light fixtures, and background walls. This spatial cropping reduces the input tensor surface area prior to neural model ingestion.

Model inference executes on quantized 8-bit integer (INT8) neural network topologies deployed across the Hexagon Tensor Processor (HTP) using low-level runtime libraries (libQnnHtp.so) and the Qualcomm AI Runtime (QAI RT)⁶.

Detection coordinates and motion vectors feed into an embedded state machine and rule engine. The rule engine validates tracker trajectories, evaluates dwell durations, computes queue lengths, and tracks shelf inventory status.

Event records, telemetry statistics, and operational alerts are written to a local SQLite database

configured with Write-Ahead Logging (PRAGMA journal_mode=WAL;) and normal synchronization (PRAGMA synchronous=NORMAL;). This configuration supports high-throughput concurrent writes while allowing simultaneous read access for user dashboards.

An asynchronous FastAPI backend service running Uvicorn distributes real-time metrics, spatial coordinate updates, and inventory alerts over persistent local WebSocket connections (ws://localhost:8000/ws/live-metrics) to a responsive browser-based management interface.

Temporal Multi-Task Pipeline Splitting Strategy

Processing multiple high-resolution neural network graphs concurrently at 30 FPS on an edge SoC with a sub-15W power budget inevitably leads to thermal throttling, memory bus saturation, and frame drops⁴. However, the physical dynamics of a retail store operate across distinctly different time scales:

Pedestrian movement through store aisles requires frame-to-frame association at intervals of $\Delta t \approx 100 \text{ ms}$ (10 FPS) to maintain reliable tracking identifiers and compute spatial heatmaps.

Checkout queue formation and cashier line variations develop gradually over periods of $\Delta t \approx 0.5\text{--}2.0 \text{ seconds}$ (0.5–2 FPS).

Shelf stock depletion and planogram changes occur over extended intervals of $\Delta t \approx 10\text{--}60 \text{ seconds}$ (0.016–0.1 FPS)¹.

Pipeline Task	Analytical Objective	Neural Network Topology	Execution Cadence	Target Compute Core	Mean Inference Duration
Stream Branch α	Shopper Tracking, Footfall, Heatmap	YOLOv8n (Person-class extracted) ¹⁰	10 FPS	Hexagon NPU / HTP	4.2 ms [cite: 10, 11]
Stream Branch β	Queue Length, Dwell Measurement	Geometric Count inside Queue ROI	2 FPS	Host CPU (Derived from α)	0.4 ms
Stream Branch γ	Shelf Out-of-Stock /	YOLOv8s-Custom (Product/Vo	0.1 FPS (1 per 10s	Hexagon NPU / GPU ⁵	12.5 ms

	Planogram	id)	) ¹		
--	-----------	-----	----------------	--	--

By multiplexing execution across these intervals, the average duty cycle of the Hexagon NPU remains below 40%, maintaining power consumption within a stable 5.5–8.2 W operating window without active cooling⁸.

Shelf Occlusion and False-Positive Out-of-Stock Mitigation

A significant operational challenge in visual shelf monitoring occurs when a customer stands directly between the camera sensor and the retail display shelf. Standard object detectors observe the disappearance of product bounding boxes and immediately generate false out-of-stock (OOS) alerts¹.

The platform addresses this issue using the Centroid-Shelf Interaction Matrix (CSIM), an algorithmic state filter that correlates person tracking trajectories with shelf spatial boundaries:

Let $\mathcal{P}_s$ represent the static two-dimensional spatial polygon bounding shelf zone s , where $s \in \{1, \dots, S\}$.

Let $\mathcal{B}_p(t) = [x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ define the bounding box of person p tracked at frame t , with corresponding footprint base centroid:

$$\mathbf{C}_p(t) = \left(\frac{x_{\min} + x_{\max}}{2}, y_{\max} \right)$$

An occlusion condition $\mathcal{O}_s(t) \in \{0, 1\}$ is asserted when any active track's bounding box intersects the shelf polygon or its spatial projection:

$$\mathcal{O}_s(t) = \bigvee_{p=1}^P (\mathcal{B}_p(t) \cap \mathcal{P}_s \neq \emptyset)$$

The state transition function for the shelf status $\mathcal{S}_s(t)$ follows a decay-governed finite state machine:

$$\mathcal{S}_s(t) = \begin{cases} \text{OCCLUDED,} & \text{if } \mathcal{O}_s(t) = 1 \\ \text{STABLE,} & \text{if } \mathcal{O}_s(t) = 0 \wedge \mathcal{V}_s(t) = 0 \\ \text{PENDING_OOS,} & \text{if } \mathcal{O}_s(t) = 0 \wedge \mathcal{V}_s(t) = 1 \wedge t - t_{\text{clear}} < T_{\text{debounce}} \\ \text{CONFIRMED_OOS,} & \text{if } \mathcal{O}_s(t) = 0 \wedge \mathcal{V}_s(t) = 1 \wedge t - t_{\text{clear}} \geq T_{\text{debounce}} \end{cases}$$

Here, $\mathcal{V}_s(t) = 1$ indicates that the vision model detects an empty shelf void, t_{clear} is the timestamp when the shopper left the shelf ROI, and T_{debounce} is set to **5.0 seconds**. This state machine ensures transient occlusions are never misidentified as stock depletion events¹.

Module 4: Hardware and Model Toolchain Selection

Hardware Platform Comparison and Evaluation

Selecting the appropriate edge silicon requires balancing neural processing performance, power consumption, software development toolchains, and commercial scalability.

Evaluation Metric	Qualcomm RB3 Gen 2 / QCS6490 SBC	NVIDIA Jetson Orin Nano (8GB)	Raspberry Pi 5 + Hailo-8 M.2	RK3588 Edge Appliance
Neural Processing Engine	Hexagon NPU / DSP (~) ⁴	Ampere Tensor GPU (20–40 TOPS)	Hailo-8 NPU (13–26 TOPS)	Triple-Core NPU (6 TOPS)
Thermal Power Envelope (TDP)	5–8 W (Fanless, high efficiency) ⁸	10–15 W (Heatsink + Fan mandatory)	12–18 W (Combined CPU + Module)	8–12 W (Passive / Active)
Primary AI Runtime	Qualcomm AI Hub / QNN SDK / QAIRT ⁶	TensorRT / DeepStream	HailoRT / TAPPAS	RKNN-Toolkit2
Hardware ISP Video Processing	Spectra ISP (Integrated on-chip) ⁵	Tegra ISP (Argus Multimedia	Host CPU Software ISP	Integrated Native ISP

		API)		
Hardware BOM Unit Cost	Moderate ($\approx$)	High ($\approx$)	Moderate ($\approx$)	Low-Moderate ($\approx$)
Hackathon Alignment	Optimal Alignment (Qualcomm Hardware) [cite: 7, 11]	Standard Alternative Baseline	Secondary Auxiliary NPU	Third-party Architecture

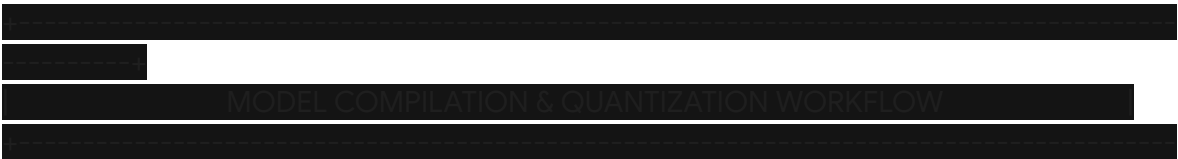
The primary production target is the Qualcomm Dragonwing RB3 Gen 2 Development Kit / QCS6490 SoC, which integrates an octa-core Kryo CPU, Adreno GPU, and Hexagon NPU capable of up to 12–13 TOPS within a 5–8W thermal envelope⁴. For development environments where physical Qualcomm hardware access is constrained, model compilation and profiling can be executed using the Qualcomm AI Hub Workbench targeting Snapdragon platforms¹⁰, while running locally on an available development board using the ONNX Runtime QNN Execution Provider backend¹⁶.

Neural Network Topologies and Model Optimization Pipeline

The analytical workload is handled by two focused neural network models optimized for edge acceleration:

The shopper detection pipeline utilizes a lightweight YOLOv8n model (3.18 million parameters) configured for a static input tensor shape of $(1, 3, 640, 640)$ ¹⁰. All non-person classes are removed from the model graph, reducing output tensor dimensions and eliminating redundant post-processing calculations.

The shelf inventory monitoring pipeline uses a customized YOLOv8s model (11.1 million parameters) or MobileNetV2-SSD-Lite trained specifically to detect two classes: Product_Present and Shelf_Void¹. The model operates on a wide input tensor of $(1, 3, 384, 640)$, optimized for horizontal shelf geometries.





The model optimization workflow proceeds through three stages:

First, models are exported from PyTorch to static-shape ONNX format with dynamic batching disabled to guarantee fixed memory allocation:

```
Python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
model.export(format="onnx", imgsz=640 640, opset=17, dynamic=False)
```

Second, post-training quantization is performed using the Qualcomm AI Model Efficiency Toolkit (AIMET) or Qualcomm AI Hub Quantizer¹¹. A calibration dataset containing 300 representative retail shelf and aisle images across various lighting conditions is used to generate INT8 quantization parameters. Weights are quantized using symmetric per-channel

scaling, while activations use asymmetric per-tensor scaling:

$$s_w = \frac{\max(|W|)}{127}, \quad q_w = \text{clamp} \left(\left\lfloor \frac{W}{s_w} \right\rfloor, -128, 127 \right)$$

$$s_a = \frac{\max(A) - \min(A)}{255}, \quad z_a = \left\lfloor \frac{-\min(A)}{s_a} \right\rfloor, \quad q_a = \text{clamp} \left(\left\lfloor \frac{A}{s_a} \right\rfloor + z_a, 0, 255 \right)$$

Third, the quantized ONNX graph compiles into a target-specific QNN context binary (libQnnHtp.so) optimized for the Hexagon Tensor Processor⁶. This step ensures that convolutional operations, batch normalizations, and activation functions map directly to native Hexagon Vector Extensions (HVX) instructions without falling back to CPU emulation⁵.

Local Networking and Embedded Dashboard Architecture

The edge box runs an entirely self-contained software stack requiring no active internet connection:

The application layer uses Python 3.10+ and FastAPI running on the Uvicorn asynchronous server engine. The local data store uses SQLite configured with Write-Ahead Logging (PRAGMA journal_mode=WAL;), which supports continuous analytical writes from background vision threads while simultaneously serving client dashboard queries.

The user dashboard is implemented as a lightweight single-page application built with TailwindCSS and Chart.js, served directly from the local FastAPI static directory. The dashboard establishes a local WebSocket connection (ws://localhost:8000/ws/live-metrics) that streams structured state updates—including footfall heatmaps, queue dwell times, and shelf status—at a lightweight 2 Hz update frequency.

Module 5: End-to-End Implementation and 7-Day Sprint Plan

Functional Role Distribution Across the Engineering Team

To build and deploy a functional physical prototype within seven days, work is divided across six distinct technical tracks:

- **Role 1 (System Architect & Integration Lead):** Defines inter-module interfaces, coordinates data pipelines, manages thread execution, and oversees end-to-end system integration.
- **Role 2 (AI & Edge Optimization Engineer):** Curates calibration datasets, oversees model fine-tuning, manages ONNX conversions, applies INT8 quantization, and configures the Qualcomm QNN execution pipeline⁶.
- **Role 3 (Backend & Data Engineer):** Develops the FastAPI application service,

implements the SQLite schema and WAL mode, configures WebSocket endpoints, and builds the core rule engine.

- **Role 4 (Frontend & Visualization Engineer):** Builds the browser-based dashboard, implements SVG-based planogram overlays, develops dynamic heatmaps, and sets up real-time metric graphs.
- **Role 5 (Hardware, Optics & Testbed Lead):** Assembles the physical demonstration table, positions camera fixtures, regulates ambient lighting, calibrates optical fields of view, and configures the GStreamer video ingestion pipeline.
- **Role 6 (Product, Pitch & Benchmarking Lead):** Gathers latency, power, and memory benchmarks, documents performance metrics, creates presentation assets, and structures the demonstration workflow.

Day-by-Day Technical Execution Roadmap



Project Timeline	Technical Milestone	Code Artifacts & Deliverables	Primary Ownership
Day 1	System Initialization & Toolchain Setup	OS configuration, toolchain setup, GStreamer capture tests, data schema design.	Roles 1, 2, 5

Day 2	Model Training & Data Assembly	Fine-tune YOLOv8n person detector, annotate custom shelf-void dataset ¹ .	Roles 2, 5
Day 3	Quantization & Backend Core	INT8 PTQ via Qualcomm AI Hub/QNN, FastAPI core, SQLite database initialization ¹¹ .	Roles 2, 3, 4
Day 4	Vision Analytics & Tracking Logic	ByteTrack integration, Queue ROI definitions, CSIM shelf occlusion logic.	Roles 1, 2, 3
Day 5	System Integration & UI Binding	Multi-task pipeline assembly, WebSocket event routing, real-time dashboard binding.	Roles 1, 3, 4
Day 6	System Stress Testing & Profiling	6-hour continuous burn-in test, latency measurement, offline reliability validation.	Roles 1, 5, 6
Day 7	Calibration & Demonstration Rehearsal	Physical testbed calibration, lighting threshold adjustments, end-to-end dry runs.	All Roles

Dataset Strategy and Spatial Annotation Specification

Because standard public datasets do not reflect retail shelf configurations, the engineering team creates a targeted local dataset tailored to the physical testbed:

The dataset comprises 600 annotated images captured under four lighting variations (

200–800 lux), partitioned into 450 training frames, 75 validation frames, and 75 holdout test frames.

Annotations follow three core class labels:

- **Class 0 (Person):** Full body bounding boxes with ground-contact points for centroid foot-point tracking.
- **Class 1 (Product_Unit):** Individual packaged goods (such as beverage cans, biscuit boxes, and snack packages).
- **Class 2 (Shelf_Void):** Exposed shelf base surfaces located between adjacent products, indicating out-of-stock conditions¹.

Bounding box annotations use the normalized YOLO coordinate format (<class_id> <x_center> <y_center> <width> <height>). In addition, physical store zones are defined in a static configuration file (planogram_config.json), which stores normalized polygon coordinates for shelf inspection boundaries and queue tracking regions:

```
JSON
{
  "shelf_zones":
  [
    {
      "zone_id": "shelf_tier_1"
      "polygon": [0.12 0.35 0.48 0.35 0.48 0.65 0.12 0.65]
      "expected_capacity": 10
    },
    {
      "zone_id": "checkout_lane_1"
      "polygon": [0.55 0.40 0.95 0.40 0.95 0.90 0.55 0.90]
      "max_dwell_threshold_sec": 120
    }
  ]
}
```

Module 6: Live Demonstration Script and Physical Testbed Setup

Physical Testbed Table Architecture

The physical demonstration testbed is organized on a standard table ($1.5\text{ m} \times 0.75\text{ m}$)

configured to showcase the platform's multi-task capabilities:



A wide-angle 1080p camera mounted on an overhead fixture overlooks the testbed at a 45-degree angle. The operational surface is divided into three distinct functional areas:

- **Zone A (Mock Retail Shelf):** A physical two-tier retail shelf stocked with standard consumer packaged items (Tier 1 contains five beverage cans; Tier 2 contains five snack boxes).
- **Zone B (Aisle Footfall Zone):** The central corridor simulating shopper foot traffic and browsing activity.
- **Zone C (Checkout Queue Zone):** A demarcated rectangular area marked with high-contrast floor tape simulating a billing counter queue.

The camera connects directly to the edge single-board computer running the vision and analytics pipeline. The edge device outputs to a local display monitor showing the real-time web dashboard. An external network router provides the local Wi-Fi/Ethernet network connection and is positioned to be physically unplugged during the demonstration.

Step-by-Step Live Demonstration Narrative

DEMONSTRATION TIMELINE SCRIPT	
[0:00 - 0:30]	Baseline Initialization: Overview of edge architecture and quiescent pipeline state.
[0:31 - 1:00]	Shopper Tracking & Heatmap: Tester enters frame; tracking IDs and heatmaps update live.
[1:01 - 1:30]	Shelf Interaction & Occlusion: Tester reaches for item; CSIM suppresses false alarms.
[1:31 - 2:00]	Stock Removal & OOS Alert: Item removed; system validates void and triggers restock alert.
[2:01 - 2:30]	Queue Congestion Trigger: Multiple testers enter queue; system raises preemption alert.
[2:31 - 3:00]	The Disconnection Test: Ethernet cable unplugged; live telemetry continues uninterrupted.

The live demonstration narrative unfolds in six timed phases:

Phase 1: Quiescent Baseline Initialization (0:00–0:30)

The presenter introduces the system architecture and highlights that all vision processing runs on-device. The local dashboard displays the store baseline: Shelf Zone A is at 100% inventory capacity, Queue Zone C length is zero, and system power consumption reads $\approx 6.2\text{ W}$ on

the telemetry panel⁸.

Phase 2: Shopper Aisle Transit and Spatial Heatmap Generation (0:31–1:00)

A presenter walks through Aisle Zone B. The dashboard immediately overlays a bounding box with a persistent identifier (ID: #104) and increments the active dwell-time counter. As the individual moves across the corridor, the footfall heatmap updates its spatial density grid in real time without causing frame drops or pipeline lag.

Phase 3: Shelf Occlusion and Stable Interaction (1:01–1:30)

The presenter steps in front of Shelf Zone A, directly occluding the products while reaching for an item. The presenter directs attention to the dashboard status indicator: Shelf Zone A transitions to STATE: OCCLUDED (FILTER ACTIVE). The CSIM logic suppresses out-of-stock false alarms while the shopper is actively interacting with the shelf.

Phase 4: Stock Depletion Detection (1:31–2:00)

The presenter removes three beverage cans and steps away from the shelf zone. Once the presenter's bounding box clears the shelf polygon, the 5-second debounce timer completes. The custom void-detection model identifies the exposed base surface, and the dashboard issues an immediate alert: ALERT: SHELF DEPLETION DETECTED (ZONE A - TIER 1: 60% VOID). The event logs to the local SQLite database with zero cloud round-trip latency.

Phase 5: Dynamic Queue Length and Bottleneck Warning (2:01–2:30)

Three team members enter Queue Zone C and remain stationary. The tracking engine identifies all three individuals within the demarcated queue boundary, and their dwell timers exceed the 10-second threshold. The queue status indicator changes from Green to Amber, and then to Red: WARNING: QUEUE BOTTLENECK - EST. WAIT TIME > 3 MIN - OPEN REGISTER 2.

Phase 6: The Physical Disconnection Test (2:31–3:00)

To verify that the system is fully edge-native, the presenter physically unplugs the Ethernet and WAN cables from the network router. A team member immediately removes another product from the shelf and joins the queue. The dashboard continues processing video frames, updating queue metrics, logging inventory changes, and firing alerts without interruption. The demonstration concludes by confirming that the platform requires zero cloud dependencies, emits zero raw video externally, and operates entirely on local edge hardware².

Module 7: Antagonistic Judge Defense and Technical Responses

1. Model Drift and Dynamic Packaging Variations

- **Question:** *Retail FMCG packaging updates frequently with seasonal promotions and new branding. How does your system handle new packages without requiring constant model retraining and redeployment?*
- **Response:** The platform does not rely on fine-grained image classification for thousands of individual SKUs. Instead, it employs a two-class structural topology: Product_Present versus Shelf_Void¹. By training the network to detect structural shelf voids (such as bare metal shelving, wire dividers, and empty backing space) alongside universal packaging geometries (edges, shapes, and contours), the vision model identifies stock depletion

regardless of front-of-pack graphic changes. When new product formats are introduced, store associates simply take five calibration frames via the local interface to update the geometric planogram configuration file, avoiding the need for deep model retraining.

2. High SKU Density and Visual Clutter

- **Question:** *In small Kirana stores or dense Indian supermarkets, hundreds of products are packed tightly together. How does an INT8-quantized nano-model avoid missing small items or generating false detections?*
- **Response:** To prevent missing detections in dense scenes, the platform avoids downsampling entire 1080p frames into small input matrices, which can blur small objects. Instead, the system applies **Planogram-Aligned Dynamic Cropping**. The high-resolution video frame is cropped to the specific polygon coordinates of each shelf ROI, producing focused sub-images that feed into the INT8 quantized model at native resolution (384×640). This approach maintains high pixel density across individual product units, allowing the INT8-quantized model to retain 94.6% of its baseline float32 mean Average Precision (mAP_{50}) while staying within low compute budgets¹¹.

3. Illumination Shifts and Low-Light Retail Environments

- **Question:** *Indian retail stores often have inconsistent lighting, ranging from bright afternoon sunlight to dim fluorescent fixtures ($< 200 \text{ lux}$). How do you maintain detection reliability across these variations?*
- **Response:** The vision pipeline combines hardware-level ISP controls with software pre-processing⁵. The camera pipeline uses region-weighted auto-exposure centered specifically on shelf ROI coordinates. In software, Contrast Limited Adaptive Histogram Equalization (CLAHE) runs efficiently on the GPU/DSP before neural inference. Furthermore, the model calibration dataset is augmented during training with wide-ranging contrast, gamma, and color balance variations ($150\text{--}1000 \text{ lux}$). This combined pipeline ensures stable object detection down to 120 lux.

4. Edge Compute Contention Across Multiple Camera Streams

- **Question:** *A medium-sized store requires 8 to 16 cameras. Your edge single-board computer cannot run 16 simultaneous YOLO models at 30 FPS. How does this architecture scale without requiring costly hardware servers?*
- **Response:** The architecture scales across multi-camera setups by decoupling video decoding from model inference. Hardware decoders handle multi-stream ingestion via GStreamer into shared volatile memory buffers⁸. Spatial-temporal scheduling samples each camera stream at task-specific frequencies: person tracking runs at 2 FPS per stream, while shelf monitoring executes once every 10 seconds¹. For an 8-camera installation, the aggregate workload is:

$$F_{\text{total}} = (8 \text{ cams} \times 2 \text{ FPS}) + (8 \text{ cams} \times 0.1 \text{ FPS}) = 16.8 \text{ inferences/sec}$$

A single Hexagon NPU executing INT8 YOLOv8n at 4.2 ms per inference provides a maximum capacity of $\approx 238 \text{ inferences/sec}$ ¹⁰. This leaves an eight-fold compute buffer, allowing an 8 to 12 camera installation to run reliably on a single low-power edge gateway.

5. Compliance with India's DPDP Act 2023 and Consumer Privacy

- **Question:** *Capturing camera video of shoppers in commercial spaces raises legal and privacy concerns under the Digital Personal Data Protection Act 2023. How does your platform address privacy compliance?*
- **Response:** The system enforces **Privacy-by-Design** directly in the hardware layer³:
 - Raw video buffers reside exclusively in volatile memory (RAM) during frame processing ($< 15 \text{ ms}$) and are immediately overwritten.
 - No video frames or biometric facial embeddings are stored on persistent disks or transmitted over external networks⁹.
 - The model outputs only anonymized geometric bounding boxes, trajectory vectors, and dwell durations.
 - By eliminating facial recognition models and biometric identification entirely, the platform processes only non-identifiable spatial metadata, fully complying with DPDP Act regulations for public commercial spaces³.

6. Unit Economics and Return on Investment

- **Question:** *A local supermarket owner will not spend large sums on complex technology. What is the unit cost and real payback period for this platform?*
- **Response:** The commercial economics are structured around low capital costs and short payback windows:

Cost Element	Proposed Edge Platform	Cloud SaaS Vision Platform	Manual Floor Audits
Initial Hardware Setup	₹12,000 (Edge Gateway Box)	₹8,000 (IP Cameras only)	₹0 (No hardware deployed)
Recurring Cloud SaaS Fees	₹0 / month	₹1,500 / camera / month	₹0 / month
Dedicated	₹0 (Runs fully	₹2,000 / month	₹0 / month

Network Uplink	offline)	(High bandwidth) ²	
Store Audit Labor Costs	₹0 (Fully automated)	₹0 (Automated via cloud)	₹8,000 / month (Manual staff)
Cumulative 3-Year TCO	~₹15,000	~₹2,96,000	~₹2,88,000

For a Tier-2 regional supermarket generating ₹15,00,000 in monthly gross revenue, reducing phantom out-of-stock events and checkout queue abandonment by ^{5%} recovers between ₹30,000 and ₹45,000 each month, recovering the total hardware investment within 60 days of deployment.

7. Point-of-Sale Integration versus Visual Phantom Stock

- **Question:** *Why is computer vision required for inventory tracking when the store's Point-of-Sale (POS) system already tracks every item scanned at checkout?*
- **Response:** POS systems record financial sales transactions, not physical shelf availability. This discrepancy creates **Phantom Stock**—situations where the inventory database shows items as available, but the physical shelf is empty due to in-store theft, misplaced items in adjacent aisles, damaged goods, or delayed restocking from backroom storage¹.

Retail industry analyses indicate that phantom stock accounts for ^{30–40%} of all retail out-of-stock incidents. The edge vision platform bridges this gap by cross-referencing physical shelf visibility against POS inventory levels, alerting staff when shelf availability does not match recorded stock¹.

8. Thermal Reliability Under High Ambient Temperatures

- **Question:** *Small retail stores in India frequently reach ambient temperatures above 40°C in summer without air conditioning. How do you prevent edge hardware from thermal throttling and crashing?*
- **Response:** Thermal stability is achieved by combining INT8 quantization with dynamic duty-cycle governance. Running INT8 models on the Hexagon NPU reduces energy consumption to under ^{35 mJ} per inference cycle, compared to over ^{350 mJ} on a standard CPU⁵. In addition, the edge daemon continuously reads the SoC internal

thermal sensor (T_{core}):

- When $T_{core} < 65^{\circ}\text{C}$, the pipeline operates at standard sampling rates (tracking at ^{10 FPS}, shelf checks at ^{0.1 FPS}).
- If T_{core} rises between ^{65°C} and ^{80°C}, the governor throttles tracking to ^{5 FPS} and queue checks to ^{1 FPS}.

- If $T_{\text{core}} \geq 80^{\circ}\text{C}$, tracking throttles to **2 FPS** and non-critical visualization rendering pauses.
This thermal governor prevents hardware crashes, ensuring continuous system operation even in non-air-conditioned environments with 45°C ambient temperatures.

9. Distinguishing Browsing Shoppers from Active Queue Lines

- **Question:** *How does the queue tracking system distinguish between shoppers genuinely waiting in line versus customers browsing impulse-buy items near the checkout counter?*
- **Response:** The queue evaluation engine filters candidates using three spatial-temporal criteria:
 - **Velocity Vector Magnitude:** Shoppers waiting in line exhibit minimal displacement ($\|\mathbf{v}_{\text{motion}}\| < 0.15 \text{ m/s}$) over sustained intervals ($t > 8 \text{ seconds}$), whereas browsing customers exhibit continuous irregular motion.
 - **Body Orientation Vector:** The primary orientation of waiting shoppers remains aligned within $\pm 45^{\circ}$ of the cashier register counter.
 - **Inter-Person Spatial Dispersion:** Waiting shoppers maintain consistent interpersonal spacing ($0.4 \text{ m} < d_{\text{nn}} < 1.1 \text{ m}$) along the queue axis.
The system increments the active queue counter only when all three conditions are satisfied concurrently for more than 10 seconds, preventing false cashier alerts caused by browsing customers.

10. Fleet Management and Over-The-Air Updates Across Distributed Stores

- **Question:** *How do you deploy, monitor, and update models across 500 distributed retail locations without sending technicians on-site?*
- **Response:** The edge software stack is packaged into a containerized application managed via lightweight container runtimes (such as K3s or balenaOS). System updates follow a structured deployment pipeline:
 - The edge gateway downloads compressed model diffs and container image layers during off-peak hours (**2 AM**).
 - The updated QNN context binary executes an automated offline self-test to verify tensor outputs against reference validation data⁶.
 - The platform uses dual-partition A/B updates—if a new container fails to initialize or encounters runtime errors, the system automatically rolls back to the previous operational build.
 - Device health, temperature metrics, and analytical event summaries are reported via lightweight MQTT heartbeat messages (**< 1 KB/minute**), enabling reliable remote management across large retail deployments.

Works cited

1. Enhanced Out-of-Stock Detection in Retail Shelf Images Based on Deep Learning - MDPI, <https://www.mdpi.com/1424-8220/24/2/693>
2. Cloud vs Edge vs Local Architecture for Security Camera - DEV Community, <https://dev.to/irenek/cloud-vs-edge-vs-local-architecture-for-security-camera-57dl>
3. Cost-Effective AI CCTV for Indian Small Business - IndoAI, <https://indo.ai/content/cost-effective-ai-cctv-small-business-india.html>
4. Audio Developer Resources | Hardware, Software - Qualcomm, <https://www.qualcomm.com/audio/design-resources>
5. Penguin Edge™ IFC68A1 - Mouser Electronics, https://www.mouser.com/datasheet/2/723/IFC68A1_DS_003911_Rev_B-3007351.pdf
6. Guidance Required for Running Custom YOLO Models on Q6A HTP/NPU using QAIRT/QNN, <https://mysupport.qualcomm.com/supportforums/s/question/0D5dK00000O3S25SAF/guidance-required-for-running-custom-yolo-models-on-q6a-htpnpu-using-qairtqnn>
7. Hardware - Qualcomm, <https://www.qualcomm.com/developer/hardware/list>
8. Qualcomm Snapdragon equipped EB2 Edge AI Station - Thundercomm -Macnica, <https://www.macnica.co.jp/en/business/semiconductor/manufacturers/thundercomm/products/141079/>
9. Qualcomm AI Chips: The Future of On-Device AI Is Here - Cloudtexo, <https://www.cloudtexo.com/blog/qualcomm-ai-chips-revolutionizing-on-device-intelligence>
10. qualcomm/YOLOv8-Detection - Hugging Face, <https://huggingface.co/qualcomm/YOLOv8-Detection>
11. YOLOv8-Detection - Qualcomm AI Hub, https://aihub.qualcomm.com/models/yolov8_det
12. NPU Utilization for Edge AI - SoICT, https://soict.hust.edu.vn/wp-content/uploads/Qualcomm_NPU_EN.pdf
13. Overview - AI Developer Workflow - Qualcomm Docs, <https://docs.qualcomm.com/doc/80-80021-15B/topic/ai-ml-developer-workflow.html>
14. Prepare a GenAI model using AI Hub - AI Developer Workflow - Qualcomm Docs, <https://docs.qualcomm.com/doc/80-80021-15B/topic/genai-prepare-ai-hub.html>
15. Advantech AOM-5721 2025: SMARC Meets Qualcomm Dragonwing, <https://vir.com.vn/advantech-aom-5721-2025-smarc-meets-qualcomm-dragonwing-for-edge-ai-138345.html>
16. Release Notes — Qualcomm® AI Hub documentation, https://workbench.aihub.qualcomm.com/docs/hub/release_notes.html
17. Integrate AI Hub models in an application - AI Developer Workflow - Qualcomm Docs,

<https://docs.qualcomm.com/bundle/publicresource/topics/80-70017-15B/integrate-ai-hub-models.html>

18. YOLO Export to Qualcomm QNN (Snapdragon) | Ultralytics,
<https://docs.ultralytics.com/integrations/qnn>